

Updated Plan: Specified Natural Flow

1. Add an explicit natural-flow mode

Add an enum to `models.py`:

```
class NaturalFlowMode(Enum):  
    ZERO = "ZERO"  
    CALCULATED = "CALCULATED"  
    SPECIFIED = "SPECIFIED"
```

Extend `InterzoneFlow`:

```
@dataclass  
class InterzoneFlow:  
    ...  
  
    natural_flow_mode: NaturalFlowMode | None = None  
    specified_natural_flow: float | None = None  
    nf_measurements: list[FlowMeasurement] = field(default_factory=list)
```

The existing `nf_measurements` field can be used for specified time-series values rather than introducing another parallel measurement model.

Mode defaults

When `natural_flow_mode` is `None`, resolve the default from the connected zone types:

Connection	Default mode
Stream ↔ Stream	CALCULATED
System gain/loss ↔ Stream	CALCULATED
Import → Stream	ZERO
Stream → Use	ZERO
Stream ↔ Storage	ZERO
Other connections	ZERO

An explicitly configured mode overrides the default.

The resolved mode should be stored in solver state or returned by a helper. It should not overwrite the user's original model.

2. Define how specified values are supplied

A `SPECIFIED` flow may obtain its daily value from either:

1. `specified_natural_flow`, for a constant value; or
2. `nf_measurements`, for a time series.

For the initial implementation, require exactly one source:

```
if mode == NaturalFlowMode.SPECIFIED:
    if specified_natural_flow is None and not nf_measurements:
        raise ValueError(...)
    if specified_natural_flow is not None and nf_measurements:
        raise ValueError(...)
```

Multiple `nf_measurements` may be summed using their adjustment factors, consistent with physical flow measurements.

A specified natural-flow value is independent of the measured physical flow. Therefore, this remains valid:

```
measured flow = 10
specified natural flow = 100
```

This is necessary for undepleted boundary-flow calculations.

3. Keep boundary natural flow separate

Do not add `available_natural` to `InterzoneFlow`, `NaturalFlowSpecification`, or any other general input model.

Boundary natural flow is a separate concept because it represents natural flow calculated outside the problem domain that may already have been apportioned or depleted upstream.

Continue supporting:

```
SolverInput.external_natural_flows
```

although it would be clearer eventually to rename it:

```
boundary_natural_flows
```

For backward compatibility, the existing name can remain during this implementation.

Internal treatment

The solver still needs to distinguish two routed values internally:

- Total natural flow.
- Natural flow available for apportionment in the current domain.

That distinction can remain in `CurFlowInfo` or in the new natural-flow result state, but it is derived solver state—not part of the general input model.

Initialization rules are:

Source	natural	available_natural
Local calculated gain	value	value
Normal specified flow	value	value
Boundary natural flow	value	0
Zero-mode flow	0	0

Thus a specified Weber–Provo Canal natural-flow component is available within the current accounting domain, while natural flow supplied from an upstream solver remains unavailable for re-apportionment.

As these values move downstream, both are routed. The difference between them continues to represent the effect of upstream diversions outside the current domain.

4. Create a focused `NaturalFlowCalculator`

Move the logic currently in `DailyDataManager.calc_natural_flows()` into a separate class.

The class does not need `measurement_beg_date` or `measurement_end_date`.

Those dates are only needed by the component that converts measurement arrays into daily values. The natural-flow calculator should receive values already resolved for the current day.

Suggested interface:

```

class NaturalFlowCalculator:
    def __init__(
        self,
        graph_manager: GraphManager,
        loss_evaluator: LossEvaluator,
    ):
        self.gm = graph_manager
        self.loss_evaluator = loss_evaluator

    def calculate(
        self,
        *,
        date: str,
        daily_flows: dict[str, CurFlowInfo],
        specified_values: dict[str, float],
        boundary_values: dict[str, float],
        loss_state: LossState | None = None,
    ) -> NaturalFlowState:
        ...

```

The current date is still needed because loss definitions may be time-varying. The calculator does not need the overall measurement range.

Responsibilities

`NaturalFlowCalculator` should:

1. Resolve each flow's effective natural-flow mode.
2. Initialize zero, specified, and boundary values.
3. Calculate natural system gains and losses.
4. Maintain natural-flow balances at stream zones.
5. Route natural flow through calculated flows.
6. Apply directional endpoint losses.
7. Enforce bidirectional and non-bidirectional sign rules.
8. Maintain internal total and available-natural values.
9. Return a complete daily natural-flow state.
10. Record which loss state or segment was used.

It should not:

- Read measurement arrays directly.
- Know the measurement date range.
- Add LP variables.
- Solve transaction priorities.
- Mutate the input graph configuration.

5. Add a daily specified-value resolver

`DailyDataManager` already knows how to map a date to measurement-array indexes. Keep that responsibility there, or move it into a small shared `DailyValueProvider`.

For example:

```
def get_specified_natural_flow_values(
    self,
    date: str,
) -> dict[str, float]:
    ...
```

For each `SPECIFIED` flow:

- Return `specified_natural_flow` when a constant is configured.
- Otherwise sum the adjusted `nf_measurements` for the day.
- Apply the same missing-measurement policy used for physical flows, unless a separate policy is deliberately introduced.

The daily orchestration becomes:

```
data_manager.set_day(date)

specified_nf = data_manager.get_specified_natural_flow_values(date)
boundary_nf = data_manager.get_boundary_natural_flow_values(date)

natural_state = natural_flow_calculator.calculate(
    date=date,
    daily_flows=data_manager.cur_flows_by_id,
    specified_values=specified_nf,
    boundary_values=boundary_nf,
    loss_state=loss_state,
)

data_manager.apply_natural_flow_state(natural_state)
```

6. Replace recursive source propagation with stream-zone balances

The existing recursive approach begins with gains and applies their impacts downstream. It becomes difficult to reason about when specified flows can be either inflows or outflows.

Instead, maintain a natural-flow balance for each stream zone.

For each stream zone, account for:

- Natural flow arriving on incoming flows.
- Natural flow leaving on specified outgoing flows.
- Natural flow arriving from specified reverse flows.
- Natural flow removed by specified reverse inflows.
- Calculated system gains or losses.
- Endpoint losses.
- The calculated downstream stream flow.

The basic balance is:

```
natural arriving
+ local natural gain
- specified natural departures
- local natural loss
= calculated routed natural outflow
```

All terms should be converted to their effect at the stream-zone boundary through shared directional-loss helpers.

Run this process in parallel for:

- `natural`
- `available_natural`

The calculation equations are the same. They differ only in their initialized source values:

- Normal local sources enter both.
- Boundary sources enter only `natural`.

7. Interpret `CALCULATED` according to connection type

`CALCULATED` does not mean exactly the same calculation for every topology.

System gain/loss ↔ stream

Calculate the natural component from the physical gain/loss value, including losses associated with the stream zone.

This preserves the existing assumption that stream gains and losses are natural flow.

The result is a fixed daily natural source or sink before downstream routing begins.

Stream ↔ stream

Calculate the natural-flow value from the stream-zone natural-flow mass balance.

This is the routed natural-flow component.

Other connections

For the initial implementation, require these to be `ZERO` or `SPECIFIED`.

Do not infer a general calculated natural-flow rule for storage or use zones until the intended natural-condition storage balance is defined.

8. Enforce directional sign rules

Use the same signed convention as physical flows:

- Positive: `from_zone → to_zone`
- Negative: `to_zone → from_zone`

Non-bidirectional flows

Natural flow must be nonnegative.

For a specified value:

```
if value < 0 and not flow.bidirectional:
    raise ValueError(
        f"Specified natural flow for {flow.id} cannot be negative "
        "because the flow is not bidirectional."
    )
```

A bad specified value should not be silently changed to zero.

For a calculated or propagated value:

```
if not flow.bidirectional:
    value = max(0.0, value)
```

This preserves the current behavior that a downstream natural-flow component cannot become negative on a one-directional stream flow.

Bidirectional flows

Keep the signed value.

A negative specified value means natural flow moves:

declared destination → declared source

The reverse flow should:

1. Leave the declared destination using the destination-side loss rule.
2. Reach the declared source using the source-side loss rule.
3. Enter the natural-flow balance of the declared source.
4. Route downstream from that source using the same rules as any other natural-flow source.

9. Centralize directional flow effects

Do not spread sign and endpoint-loss logic throughout the calculator.

Add helpers that answer how a signed value affects a particular zone:

```
def get_flow_departure_at_zone(
    flow: InterzoneFlow,
    value: float,
    zone_id: str,
    *,
    date: str,
    loss_state: LossState | None,
) -> float:
    ...

def get_flow_arrival_at_zone(
    flow: InterzoneFlow,
    value: float,
    zone_id: str,
    *,
    date: str,
    loss_state: LossState | None,
) -> float:
    ...
```


The helpers should understand:

- Positive versus negative direction.
- `loss_from_zone`.
- `loss_to_zone`.
- Whether the requested zone is the actual source or destination.
- Constant fractions.
- Future piecewise-linear loss segments.

This avoids reproducing expressions such as:

```
value * (1 - flow.loss_to_zone)
```

throughout `solver.py`.

10. Introduce a shared loss-evaluation abstraction now

To remain compatible with the planned piecewise-linear loss work, the new natural-flow code should never treat a loss definition as a numeric fraction directly.

Do not write:

```
flow_after_loss = flow_before_loss * (1 - flow.loss_to_zone)
```

Instead, introduce an interface such as:

```
class LossEvaluator:
    def resolve_endpoint_loss(
        self,
        *,
        flow: InterzoneFlow,
        side: Literal["from_zone", "to_zone"],
        date: str,
        driver_flow: float,
        loss_state: LossState | None = None,
    ) -> ResolvedLossRelation:
        ...
```

The returned relation should contain enough information for both constant and piecewise-linear losses:

```

@dataclass(frozen=True)
class ResolvedLossRelation:
    segment_index: int | None
    min_driver_flow: float
    max_driver_flow: float | None
    loss_slope: float
    loss_intercept: float

    def loss_at(self, flow: float) -> float:
        return self.loss_slope * flow + self.loss_intercept

    def remaining_at(self, flow: float) -> float:
        return flow - self.loss_at(flow)

```

A constant fraction (r) becomes:

```

loss_slope = r
loss_intercept = 0

```

A piecewise-linear loss segment becomes:

```

loss_slope = segment slope
loss_intercept = segment intercept

```

This prevents the natural-flow implementation from having to be rewritten when variable losses are activated.

11. Distinguish total-flow and component-loss operations

A piecewise-linear absolute-loss curve cannot always be applied to a component by multiplying that component by an average fraction.

The shared loss service should eventually expose two distinct operations:

```

def transform_total_flow(...):
    """Apply the complete affine loss relation to a total flow."""

def transform_component_increment(...):
    """Apply the active marginal relation to one accounting component."""

```

Natural flow and available natural are accounting components of total physical flow. Their loss allocation should use the same component-increment policy eventually used by transaction components.

For the current constant-loss implementation, both operations reduce to the existing proportional calculation.

Adding the distinction now avoids embedding an assumption that fails when a segment has a nonzero intercept.

12. Make natural-flow calculation dependent on active loss state

The future loss algorithm will update active loss segments while a transaction is being maximized.

Therefore, natural-flow calculation should be treated as:

```
natural state =  
    function(date, daily physical flows, specified values, boundary values,  
        active loss state)
```

It should not be considered permanently fixed for the entire day when an allocation-dependent loss segment can change.

`NaturalFlowState` should record the version or identity of the loss state used:

```
@dataclass  
class NaturalFlowState:  
    flows: dict[str, NaturalFlowComponents]  
    zone_available: dict[str, float]  
    loss_state_version: int | None  
    loss_segments_used: dict[str, int]
```

When the future `LossStateManager` advances a segment:

1. Recalculate affected natural-flow balances.
2. Update the natural-flow LP constraint bounds.
3. Continue solving the same priority objective.
4. Do not treat the internal breakpoint as the transaction's final limit.

Initially, while losses remain constant fractions, natural flow will still be calculated only once per day.

13. Return state instead of relying on mutation

Suggested internal structures:

```
@dataclass
class NaturalFlowComponents:
    natural: float = 0.0
    available_natural: float = 0.0

@dataclass
class NaturalFlowState:
    by_flow_id: dict[str, NaturalFlowComponents]
    available_at_zone: dict[str, float]
    loss_state_version: int | None = None
    loss_segments_used: dict[str, int] = field(default_factory=dict)
```

Again, `available_natural` here is internal computational state. It is not a user-configurable property of an interzone flow.

For compatibility, `DailyDataManager.apply_natural_flow_state()` can copy the values into the existing `CurFlowInfo` fields.

Later, `Apportioner` can consume `NaturalFlowState` directly.

14. Determine a natural-flow dependency order

Before calculating a day, build a dependency graph for calculated stream-to-stream natural flows.

A calculated stream flow depends on:

- Fixed or specified inflows to its source zone.
- Calculated upstream stream flows.
- Fixed specified departures.
- The calculated local gain/loss value.

Process stream zones in topological order where possible.

Ambiguous branches

If a stream zone has two calculated downstream stream flows:

```
B → C
B → D
```

natural-flow mass balance determines only their sum.

Until a split rule is implemented, require at most one calculated routed outflow per stream zone. Other departures must be `ZERO` or `SPECIFIED`.

Raise a clear error such as:

```
Natural flow at zone B is underdetermined because both B>C and B>D
use CALCULATED mode.
```

Negative specified flows

A negative specified bidirectional flow creates a fixed contribution to the declared source zone. It does not itself create an unknown dependency, so it can be applied before processing calculated downstream flows.

Cycles

Detect cycles involving calculated routed flows and reject them unless a future simultaneous natural-flow balance solver is introduced.

15. Update the LP natural-flow constraints

`Apportioner.apply_nf_mass_balance_constraints()` currently reconstructs availability by examining only inflows from stream and system-gain/loss zones.

That logic would omit specified imports, storage releases, reverse flows, and other explicitly natural components.

Replace it with the result calculated for the stream zone:

```
nf_available = natural_flow_state.available_at_zone[z.id]
```

Then apply:

```
self.engine.update_constraint_lb(
    name=PREFIX_NF_ZONE + z.id,
    lb=0,
)
```

```
self.engine.update_constraint_ub(  
    name=PREFIX_NF_ZONE + z.id,  
    ub=max(0, nf_available),  
)
```

For now, transaction natural-flow availability remains nonnegative even if some bidirectional interzone-flow natural components are signed.

When active loss segments change in the future, this same method can be called again with an updated `NaturalFlowState`.

16. Preserve boundary-flow behavior

The current boundary tests should continue to work unchanged.

For example:

```
A>B measured flow = 10  
A>B boundary natural flow = 100  
local natural gain at B = 10
```

The downstream result remains:

```
total natural downstream = 110  
available natural downstream = 10
```

The boundary natural flow is routed and reported, but is not made available to downstream transactions.

A normal specified natural flow behaves differently:

```
IMPORT>B specified natural flow = 5
```

This produces:

```
natural contribution = 5  
available-natural contribution = 5
```

because it originates within the current accounting domain rather than representing water already accounted for upstream.

17. Output changes

Natural flow should eventually be returned directly rather than only through synthetic `_NF` and `_CPI` apportionments.

Add:

```
@dataclass
class SolverOutputNaturalFlow:
    date: str
    interzone_flow_id: str
    mode: NaturalFlowMode
    natural_flow: float
    is_boundary_flow: bool
```

Do not expose `available_natural` in the general natural-flow output unless it is specifically useful for diagnosing a boundary.

A separate boundary diagnostic may report:

```
@dataclass
class SolverOutputBoundaryNaturalFlow:
    date: str
    interzone_flow_id: str
    natural_flow: float
    available_within_domain: float
```

The existing synthetic outputs can remain during migration.

18. Testing plan

Mode and default tests

1. Stream-to-stream defaults to `CALCULATED`.
2. System-gain/loss-to-stream defaults to `CALCULATED`.
3. Import-to-stream defaults to `ZERO`.
4. Stream-to-use defaults to `ZERO`.
5. Explicit mode overrides the topology default.

Specified-value tests

1. Constant specified import enters downstream natural flow.
2. Time-series specified import uses the correct daily value.

3. Multiple adjusted natural-flow measurements are summed.
4. A specified diversion reduces routed downstream natural flow.
5. A specified reservoir outflow enters downstream natural flow.
6. A specified value may exceed the measured physical flow.

Boundary tests

1. Existing `external_natural_flows` tests remain unchanged.
2. Boundary natural flow is routed but is not available for apportionment.
3. Local natural gain below a boundary remains available.
4. Boundary natural flow remains unavailable during the spill pass.

Direction tests

1. Negative specified value on a non-bidirectional flow raises an error.
2. Calculated natural flow on a non-bidirectional flow is limited to zero.
3. Negative specified value on a bidirectional flow reverses direction.
4. Reversed flow uses the correct endpoint-loss sides.
5. Reversed natural flow enters the declared source zone and routes downstream.

Network tests

1. One calculated downstream flow is determined by zone balance.
2. Two calculated downstream branches raise an underdetermined-flow error.
3. Calculated-flow cycles raise a clear error.
4. A specified branch plus one calculated branch is solvable.

Loss-compatibility tests

1. Constant-loss results match existing behavior.
2. Natural-flow code never performs arithmetic directly on `LossDefinition`.
3. Directional helpers correctly resolve `loss_from_zone` and `loss_to_zone`.
4. A resolved affine loss segment preserves its intercept.
5. Natural-flow state records the loss segment used.
6. Recalculating with a different active segment updates the NF constraint.
7. Boundary-unavailable and locally available components remain distinct after loss routing.
8. A loss breakpoint is treated as an internal relinearization event rather than a final allocation limit.

19. Recommended implementation sequence

Phase 1: Loss-compatible refactor

1. Add `NaturalFlowMode`.
2. Add default-mode resolution.
3. Introduce `LossEvaluator` and directional endpoint helpers.
4. Implement constant fractions through `ResolvedLossRelation`.
5. Replace direct natural-flow loss arithmetic with the shared helpers.

6. Keep existing behavior and tests unchanged.

Phase 2: Extract the calculator

1. Add `NaturalFlowCalculator`.
2. Move current natural-flow logic out of `DailyDataManager`.
3. Remove measurement range arguments from natural-flow calculation.
4. Add `NaturalFlowState`.
5. Continue copying results into `CurFlowInfo` temporarily.
6. Preserve all boundary tests.

Phase 3: Add specified positive values

1. Use `specified_natural_flow` and `nf_measurements`.
2. Support imports, diversions, storage connections, and explicit stream flows.
3. Include specified flows in stream-zone balances.
4. Use calculated zone availability in the LP constraints.

Phase 4: Add signed bidirectional values

1. Validate negative specified values.
2. Apply reverse directional losses.
3. Route negative bidirectional values from the actual receiving zone.
4. Clamp calculated non-bidirectional values at zero.
5. Add dependency and cycle validation.

Phase 5: Integrate active loss segments

1. Add affine segment information to `ResolvedLossRelation`.
2. Pass `LossState` into natural-flow calculation.
3. Record segment dependencies in `NaturalFlowState`.
4. Recalculate affected natural-flow constraints after segment transitions.
5. Use the same component-loss transformation for natural flow and transaction components.

Phase 6: Output cleanup

1. Add direct natural-flow output records.
2. Add boundary-specific diagnostics.
3. Retain synthetic `_NF` and `_CPI` records for backward compatibility.
4. Eventually rename `external_natural_flows` to `boundary_natural_flows`.

Core design rules

1. `available_natural` is internal solver state, not a general flow configuration.
2. Only boundary natural flow initializes as unavailable.
3. Normal specified natural flow is available within the current domain.

4. `NaturalFlowCalculator` receives daily resolved values and does not manage measurement date ranges.
5. Negative values are allowed only on bidirectional flows.
6. Calculated natural flow is clamped to zero on non-bidirectional flows.
7. All endpoint-loss logic goes through one direction-aware loss service.
8. Natural-flow code must operate on affine loss relations, not assume every loss is a constant fraction.
9. Natural-flow constraints must be refreshable when an active loss segment changes.
10. Specified and calculated natural flow use the same downstream-routing logic after their initial values are established.